

Архитектура на системата — текущо състояние

Основен принцип

Системата няма да бъде направена само за програмиране или конкретна област. Тя трябва да може да се използва за:

- програмиране;
- езици;
- математика;
- бизнес;
- други видове обучения.

Тоест **областта на обучение не е част от основната архитектура.**

V1 — Основна версия

V1 представлява **общата картина на обучителната система.**

До момента сме завършили три основни концепции:

v1
|
├─ Student / Студент 
├─ Teacher / Преподавател 
└─ Course / Курс 

1. Student / Студент

За студента трябва да можем да виждаме какво се случва с неговото обучение по конкретен курс.

Информация за курса

Студентът може да бъде:

- записан;
- обучаващ се;
- завършил;
- отказал се/прекратил.

Progress / Напредък

Във V1 е обобщена стойност:

7 / 10
70%

и можем да покажем:

Remaining:
3 / 10

Attendance / Присъствие

Отделно от прогреса:

7 / 10

70%

Това е важно:

Progress $\neq$ Attendance

Студентът може например да има:

Progress: 70%

Attendance: 90%

Оценки

Пазим:

Intermediate Grade / Междинна оценка

Final Grade / Крайна оценка

Във V1 те са **обобщени стойности**.

В V2 ще можем да изградим детайлна система за оценяване и част от тези стойности да станат изчисляеми.

2. Teacher / Преподавател

Преподавателят е самостоятелна концепция.

Не го обвързваме директно с един курс.

Избрахме:

Teacher



TeacherCourseInstance



CourseInstance

Тоест:

- един преподавател може да води много провеждания;
- едно провеждане може да има повече от един преподавател.

Пример:

Иван

- └─ C# — 2026
- └─ Python — 2026
- └─ C# Advanced — 2027

Мария

- └─ C# — 2027
- └─ JavaScript — 2027

Какво НЕ правим във V1

Не записваме още:

„Иван е преподавал тема LINQ.“

Това остава за V2, когато въведем теми и занятия.

3. Course / Курс

Тук направихме най-много уточнения.

Основната идея е:

V1 Course е визитна карта на курса + обща информация за конкретното му провеждане и студентите.

Course / Курс

Самият курс съдържа обща информация:

Name / Име

Code / Код

Description / Описание

Requirements / Изисквания

Level / Ниво

Language / Език

TotalTopics / Общ брой теми

Във V1 **не описваме самите теми**.

Например:

C# Основи

Total Topics: 10

Но не:

1. Variables

2. Classes

3. LINQ

...

Това е V2.

CourseInstance / Провеждане

Разделихме **Course** от конкретното му провеждане.

Например:

Course:

C# Основи

↓

CourseInstance:

C# Основи — 2026

Така един курс може да бъде провеждан многократно.

Например:

C# Основи

├

Провеждане 2026

├

Провеждане 2027

├

Провеждане 2028

И всяко провеждане може да има различна организация.

Начин на провеждане

Имаме три основни възможности:

Online / Онлайн

On-site / Присъствен

Hybrid / Смесен

Но вземем важно решение:

Hybrid не е задължително отделен специален тип данни.

При смесено обучение можем да имаме:

Online
Capacity: 20

On-site
Capacity: 10

и системата разбира, че това е Hybrid.

Това позволява статистика по начин на участие.

Capacity / Капацитет

При смесено обучение потребителят може да определи отделни капацитети:

Online: 20
On-site: 10

Total: 30

Това е по-добре от:

HybridCapacity: 30

защото знаем **как са разпределени местата**.

Дни и период на деня

Във V1 решихме да пазим:

Дни

Например:

Monday / Понеделник

Wednesday / Сряда

Част от деня

Morning / Сутрин

Afternoon / Следобед

Evening / Вечер

Но **точните часове** остават за V2.

Година и място

За конкретното провеждане във V1 можем да знаем:

Year:
2026

City:
Vidin / Видин

ако обучението е присъствено или смесено.

Не навлизаме още в:

Building
Room
Address

освен ако по време на реализацията не решим, че са необходими.

Какво нарочно оставихме за V2

Това е много важно, защото вече имаме ясна граница между версиите.

V2 — Детайли на обучението

Ще добавим:

Subject / Предмет

↓
Topic / Тема

↓
Lesson / Занятие

Тогава например:

Total Topics = 10

от V1 ще се превърне в реални 10 теми.

И:

Progress = 7 / 10

вече може да бъде **изчислен**, вместо просто въведен.

V2 — Детайлен график

Във V1:

Monday

Wednesday

Evening

Във V2:

Monday

18:00 – 20:00

Wednesday

18:00 – 20:00

и конкретните занятия могат да имат реални дати и часове.

V2 — Детайлно преподаване

Във V1:

Teacher → CourseInstance

Във V2 можем да имаме:

Teacher → Lesson

и да знаем:

кой преподавател е водил конкретната тема/занятие.

V2 — Детайлно присъствие

Във V1:

Attendance:

7 / 10

70%

Във V2:

Lesson 1 → Present

Lesson 2 → Present

Lesson 3 → Absent

...

и системата изчислява:

7 / 10

70%

V2 — Детайлни оценки

Във V1:

Intermediate: 5.25

Final: 5.75

Във V2 можем да имаме:

Quiz

Assignment

Practical exam

Midterm

Final exam

и съответните оценки да се използват за изчисляване на междинната и крайната оценка.

Най-важният архитектурен принцип, който излезе

Мисля, че това е едно от най-силните ни решения досега:

V1 описва общото състояние. V2 добавя реалните детайли, от които това състояние може да бъде изчислено.

Например:

V1

Progress: 7 / 10

Attendance: 8 / 10

↓

V2

10 реални теми

10 реални занятия

8 присъствия

7 завършени

↓

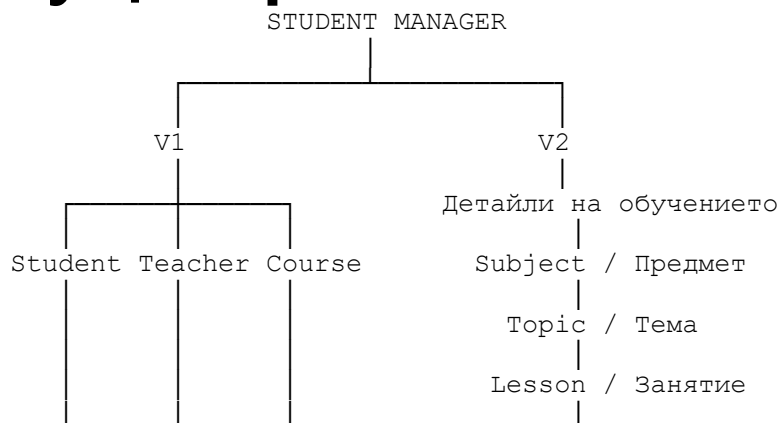
Calculated

Progress: 70%

Attendance: 80%

Remaining: 3

Текуща карта



- може да включва/изключва достъпа до определени функции/форми;
- според правата системата може да **показва или скрива съответните менюта и линкове**;
- реалната проверка на правата се извършва от системата, а не само чрез скриване на UI;
- не получава автоматично техническите права на System Administrator.

V2 ⌚

Оставяме:

- индивидуални права за конкретен потребител;
- изключения от правата на ролята;
- по-сложна система за наследяване на права;
- евентуално временни права.

Така V1 остава:

```
User
  ↓
Role
  ↓
Permissions
  ↓
Access
```

а V2 може да стане:

```
User
  ↓
Role
  ↓
Role Permissions
  +
User-specific Permissions
  ↓
Final Access
```

Administrator V1 → приет. ✓

Вече имаме изчистени **4 от 5** роли:

System Administrator	✓
Administrator	✓
Manager	⌚
Teacher	✓
Student	✓

Остава ни само **Manager / Мениджър**, след което можем да направим цялостна проверка на петте роли и да видим дали някъде има припокриване или липсващо право.

3. Вие казахте:

за manager оставам както ти предложи

4. ChatGPT каза:

Да, тогава го фиксираме така. 👍

Manager / Мениджър — V1

Основната му задача е **оперативното управление на курсовете и участниците в тях.**

Course / Курс

Manager може да:

- създава курс;
- редактира курса;
- определя:
 - име;
 - описание;
 - изисквания;
 - ниво;
 - език;
 - общ брой теми;
 - година;
 - капацитет;
 - начин на провеждане;
 - дни;
 - сутрин / следобед / вечер.

Teachers / Преподаватели

Manager:

- избира преподаватели;
- назначава преподаватели към курс;
- премахва преподаватели от курс;
- вижда кои преподаватели са назначени към даден курс.

Students / Студенти

Manager:

- вижда студентите в управляваните курсове;
- одобрява/отхвърля заявки за записване;
- записва студент;

- премахва студент от курс;
- вижда общия прогрес;
- вижда присъствието;
- вижда оценките.

Но:

Manager не променя оценките и присъствието — това остава отговорност на Teacher.

Какво не може Manager

System Status / Maintenance
→ System Administrator

Roles / Permissions
→ Administrator

Teaching / Attendance / Grades
→ Teacher

Така получаваме много чисто разделение:

System Administrator
→ Системата

Administrator
→ Достъпът

Manager
→ Организацията

Teacher
→ Обучението

Student
→ Участието

V1 — Петте роли вече са изчистени

1. System Administrator
2. Administrator
3. Manager
4. Teacher
5. Student

Според мен **не трябва да добавяме повече права към Manager засега**. По-добре следващата ни стъпка да бъде да направим **пълна матрица „Role x Permission“ за V1**, за да видим на едно място дали има конфликт или дупка в архитектурата.

5. Вие казахте:

ОК

6. ChatGPT каза:

Супер. 🍑 Тогава следващата точка е **матрицата на ролите и правата за V1**.

Преди да я направим окончателна, бих я подредил по функционалности, а не по таблици. Така много по-лесно ще видим припокриванията:

Функционалност	System Admin	Admin	Manager	Teacher	Student
System Status	✓	👁	👁	👁	👁
Maintenance	✓	✗	✗	✗	✗
System Announcement	✓	✗	✗	✗	✗
Управление на роли	✗	✓	✗	✗	✗
Управление на права	✗	✓	✗	✗	✗
Създаване на курс	✗	според правата	✓	✗	✗
Редактиране на курс	✗	според правата	✓	✗	✗
Назначаване на преподавател	✗	според правата	✓	✗	✗
Записване на студент	✗	според правата	✓	✗	собствено
Присъствие	✗	според правата	👁	✓	собствено
Оценки	✗	според правата	👁	✓	собствени
Собствен профил	✗	собствен	собствен	собствен	собствен
UI настройки	✗	собствен	собствен	собствен	собствен
Курсове	системен статус	управление според права	управление	свои налични/свои	
Статистика	✗	според права	свои курсове	свои курсове	разрешена

Тук има една важна точка за обсъждане: Administrator не трябва да бъде автоматично „може всичко“. Вече решихме, че той **определя правата**, затова в матрицата неговият достъп до конкретните функции може да зависи от зададените му permissions.

A Manager, Teacher и Student имат основните си фиксирани функции. Следващата стъпка бих я направил **точно тази таблица да я минем ред по ред**, за да не остане някое право неуточнено.

7. Вие казахте:

може ли да направиш като таблица функционалност, другите колони всички роли

8. ChatGPT каза:

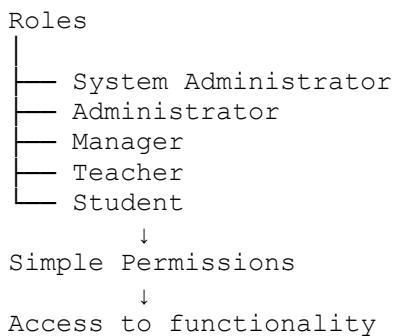
Да. Така ще е много по-удобно за преглед. Ще използвам:

- ✓ — има право
- 👁 — само вижда
- — — няма достъп
- **собствен** — само собствените данни
- **според права** — зависи от Permission, зададен от Administrator

V1 — Role × Functionality

Функционалност	System Administrator	Administrator	Manager	Teacher	Student
System Status / Състояние на системата	✓	👁	👁	👁	👁
Maintenance Mode / Поддръжка	✓	—	—	—	—
Maintenance Message / Съобщение при поддръжка	✓	—	—	—	—
System Announcement / Системно съобщение	✓	—	—	—	—
Roles / Управление на роли	—	✓	—	—	—
Permissions / Управление на права	—	✓	—	—	—
User Profile / Собствен профил	собствен	собствен	собствен	собствен	собствен
UI Settings / UI настройки	собствен	собствен	собствен	собствен	собствен

Функционалност	System Administrator	Administrator	Manager	Teacher	Student
Create Course / Създаване на курс	—	според права	✓	—	—
Edit Course / Редактиране на курс	—	според права	✓	—	—
View Course / Преглед на курс	—	според права	✓	свои	✓
Course Capacity / Капацитет	—	според права	✓	👁	👁
Course Type / Начин на провеждане	—	според права	✓	👁	👁
Course Year / Година	—	според права	✓	👁	👁
Assign Teacher / Назначаване на преподавател	—	според права	✓	—	—
Remove Teacher / Премахване на преподавател	—	според права	✓	—	—
View Students / Преглед на студенти	—	според права	✓	свои курсове	статистика
Enroll Student / Записване на студент	—	според права	✓	—	собствено
Approve Enrollment / Одобряване на записване	—	според права	✓	—	—
Remove Student / Премахване от курс	—	според права	✓	—	—
Attendance / Присъствие	—	според права	👁	✓	собствено
Grades / Оценки	—	според права	👁	✓	собствени
Progress / Прогрес	—	според права	👁	✓	собствен
Course Statistics / Статистика за курс	—	според права	✓	✓	разрешена
Other Student Profiles / Профили на други студенти	—	според права	✓	свои курсове	—



Ако в процеса на разработката установим, че:

- дадено право липсва;
- дадено право е излишно;
- две права трябва да се обединят;
- някоя роля трябва да получи/загуби достъп;
- дадена функционалност всъщност е за V2;

просто променяме модела.

Основното правило

Не проектираме V1 така, сякаш знаем предварително как ще изглежда V3.

Правим достатъчно добра основа, върху която после можем да надграждаме.

Това пасва много добре и на решенията ни досега:

- **V1** → Course / Курс;
- **V2** → повече детайли + Seminar / Семинар;
- **V3** → евентуално независими модули;
- ролите остават прости;
- permissions могат да се разширяват;
- UI настройките са отделени;
- бъдещите функции не се натъпкват преждевременно във V1.

Това бих го записал като архитектурен принцип на проекта:

Simple → Clean → Flexible → Extensible

Просто → Изчистено → Гъвкаво → Разширяемо

И според мен можем да продължим към следващата точка от архитектурата.

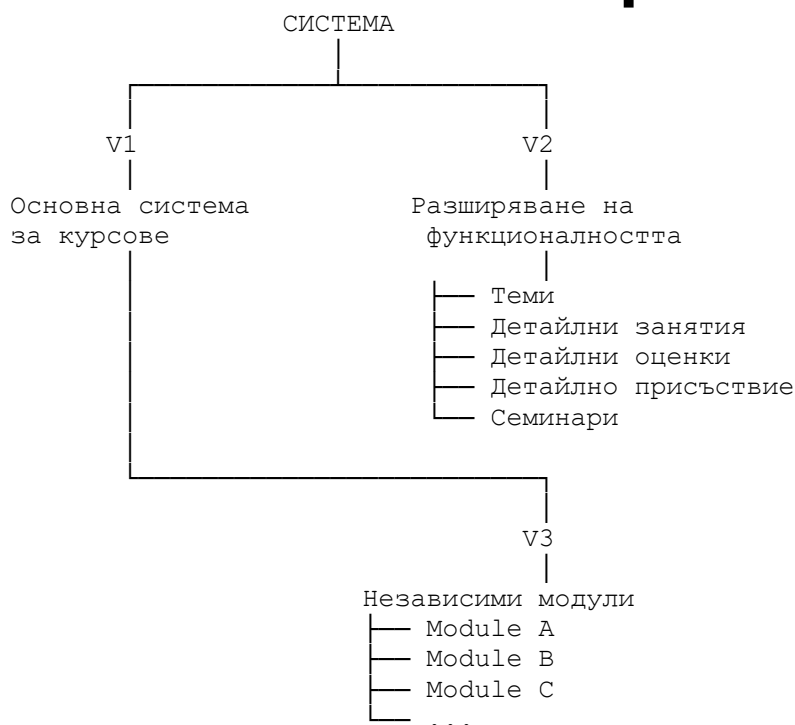
3. Вие казахте:

може ли да направиш обобщение на плана, архитектура и таблици, блок схеми

4. ChatGPT каза:

Да. Вече имаме достатъчно решения, за да направим **обща V1 картина**. Ще я разделя на: план, архитектура, роли, таблици и блок-схеми. Там, където още не сме взели окончателно решение, ще го отбележа като **за обсъждане**, вместо да го измислям.

1. Основен план на проекта

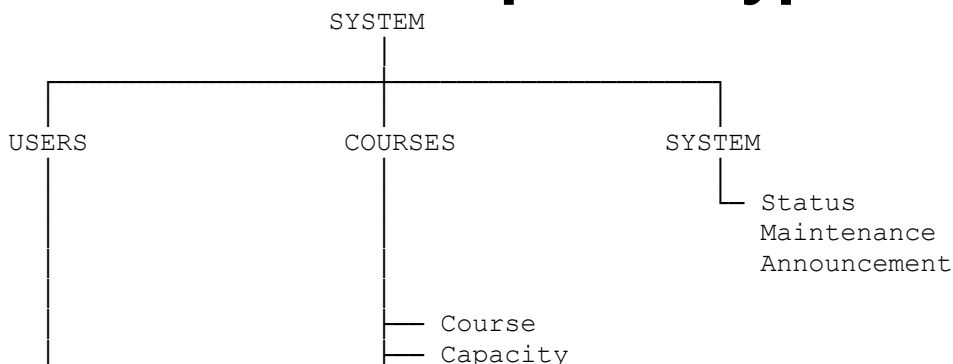


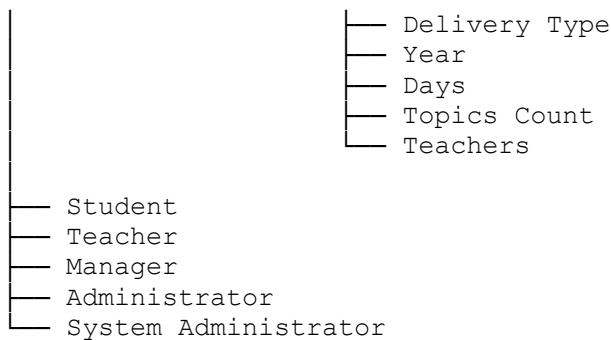
Основният принцип:

V1 → проста, изчистена и гъвкава основа.

Не изграждаме предварително всичко за V2/V3.

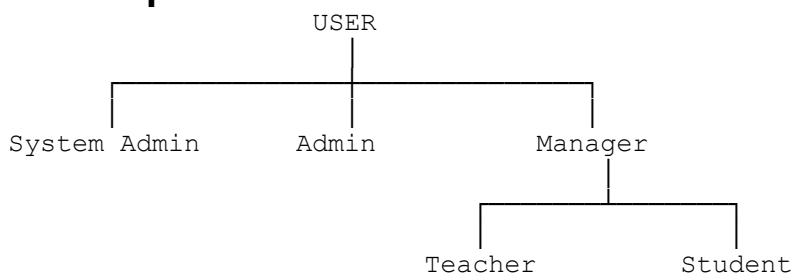
2. V1 — Основна архитектура





3. Роли — V1

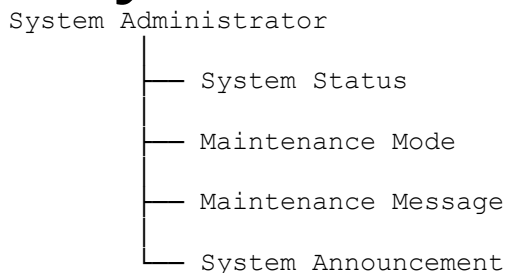
Имаме 5 роли:



Функционално ги разграничихме така:

Роля	Основна отговорност
System Administrator	Състояние и поддръжка на системата
Administrator	Роли, права и достъп
Manager	Управление на курсове и участници
Teacher	Обучение, присъствие и оценки
Student	Участие и собствено обучение

4. System Administrator



V1

Само базова системна администрация.

V2/V3

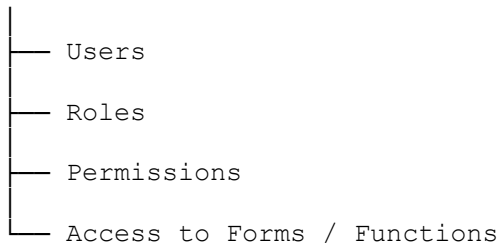
Оставяме за по-късно:

Logs
 Monitoring
 Advanced Configuration
 Module Management
 Independent Updates

5. Administrator

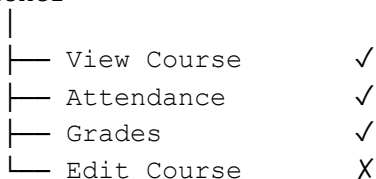
Основната му функция е **управление на достъпа**.

Administrator



Например:

Teacher



UI:

Permission = TRUE

↓

показва се функцията

Permission = FALSE

↓

скрива се функцията

+

съвърхът блокира достъпа

Важно: скриването на линк не е механизъм за сигурност. Правата трябва да се проверяват и на ниво backend.

V2

Оставяме:

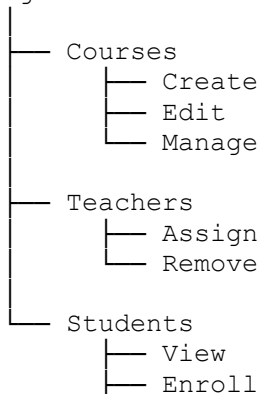
Role Permissions

+

Individual User Permissions

6. Manager

Manager



- Approve
- Reject
- Remove

Manager определя основните параметри на курса:

- Course
 - Name
 - Description
 - Requirements
 - Level
 - Language
 - Topics Count
 - Year
 - Capacity
 - Delivery Type
 - Days / Time period

Teacher **не се назначава сам.**

- Manager
 - Assign Teacher
 - ↓
 - Course

Manager вижда учебната информация, но **не въвежда оценките и присъствието.**

7. Teacher

- Teacher
 - My Profile
 - UI Settings
 - My Courses
 - My Students
 - Attendance
 - Grades

Teacher работи само с курсовете, към които е назначен.

V1

Присъствие:

7 / 10
70%

Оценки:

Intermediate Grade
Final Grade

V2

Ще можем да преминем към:

- Course
 - ↓
 - Topic
 - ↓
 - Lesson

↓
Teacher

и да определяме **кой преподавател е водил конкретна тема/заяние**.

8. Student

Student

- My Profile
- Course Catalog
- Enrollment
- My Courses
- Progress
- Attendance
- Grades
- UI Settings

Студентът вижда:

Course

- Status
- Progress
- Remaining
- Attendance
- Intermediate Grade
- Final Grade

Например:

Progress:

7 / 10 — 70%

Attendance:

8 / 10 — 80%

Поверителност

Student A

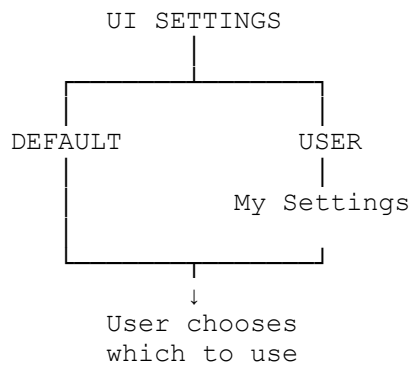
- Own Profile → FULL
- Own Data → FULL
- Other Students
 - Aggregated Statistics ONLY

Не вижда личните профили, оценки, присъствие или индивидуален прогрес на други студенти.

9. UI Settings

Това е отделна концепция, която важи за потребителите.

Имаме:



Default

Системно зададени настройки.

User

Личните настройки на потребителя:

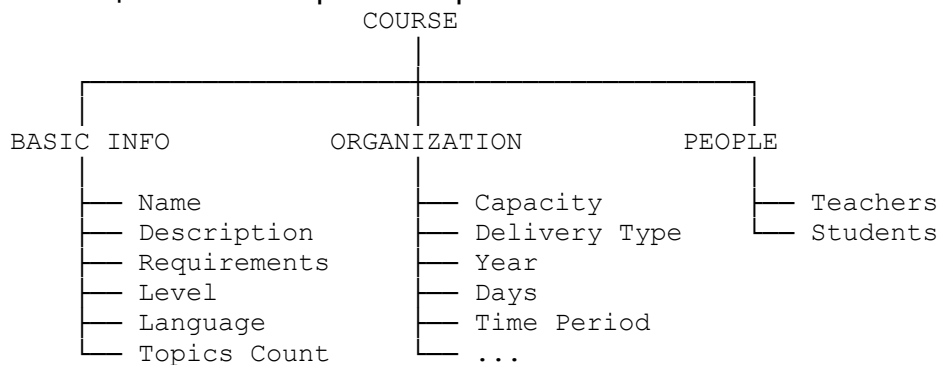
Theme
Language
Menu
Element Size
Layout
Other Visual Preferences

И двете конфигурации **се запазват**.

Потребителят може да превключва между тях.

10. Course — V1

Тук е същината на първата версия.



Важно:

V1 има само обща информация за темите.

Например:

Topics Count = 10

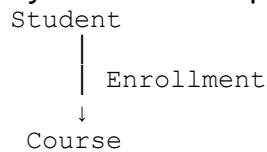
а не:

Topic 1
Topic 2
Topic 3
...

Детайлите са за V2.

11. Student ↔ Course

Тук имаме свързващата логика:



Един студент може да има:

Course A → Completed
Course B → In Progress
Course C → Enrolled

А за текущ курс:

Progress:
7 / 10 — 70%

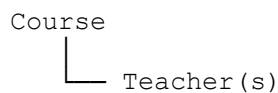
Attendance:
8 / 10 — 80%

Intermediate:
5.25

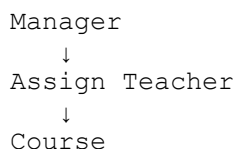
Final:
—

Това е важна част от модела.

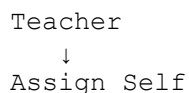
12. Course ↔ Teacher



Назначаването:

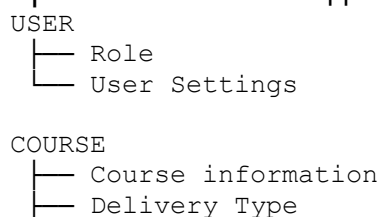


Не:



13. Основни таблици — V1

На база на всичко, което сме обсъдили, бих започнал с приблизително следната структура:



- └─ Course status/information
- TEACHER
 - └─ User relationship
- STUDENT
 - └─ User relationship
- COURSE_TEACHER
 - └─ Course ↔ Teacher
- COURSE_STUDENT
 - └─ Course ↔ Student
- ATTENDANCE
 - └─ Student ↔ Course
- GRADES
 - └─ Student ↔ Course

Тук **не бих фиксира**л окончателно имената и броя на таблиците **още**, защото предстои да разгледаме реалния V4.txt и текущата структура на проекта.

14. Какво НЕ слагаме във V1

Това е също толкова важно.

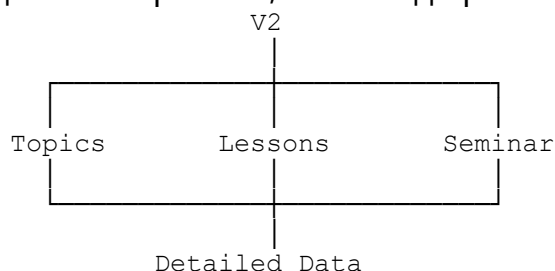
V1

- └─ ✗ Detailed Topics
- └─ ✗ Detailed Lessons
- └─ ✗ Teacher per Topic
- └─ ✗ Seminar
- └─ ✗ Individual Permissions
- └─ ✗ Independent Modules
- └─ ✗ Module Fault Isolation
- └─ ✗ Advanced Monitoring
- └─ ✗ Complex Scheduling

Те не са забравени — **умишлено са оставени за бъдещи версии.**

15. V2

След като V1 работи, можем да развием:



Тогава числата от V1 могат да станат **изчисляеми**.

Например вместо:

7 / 10

70%

системата реално да изчислява:

Completed Topics

/

Total Topics

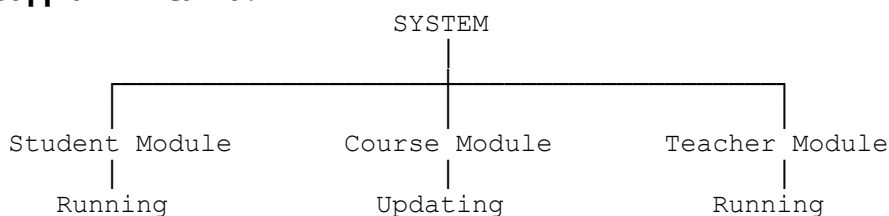
↓

Percentage

Същото за присъствие и оценки.

16. V3 — евентуално

Тук е идеята, която обсъдихме, но **не я заключваме като задължителна**:



Ако:

Course Module

↓

Problem

останалите:

Student Module → Running

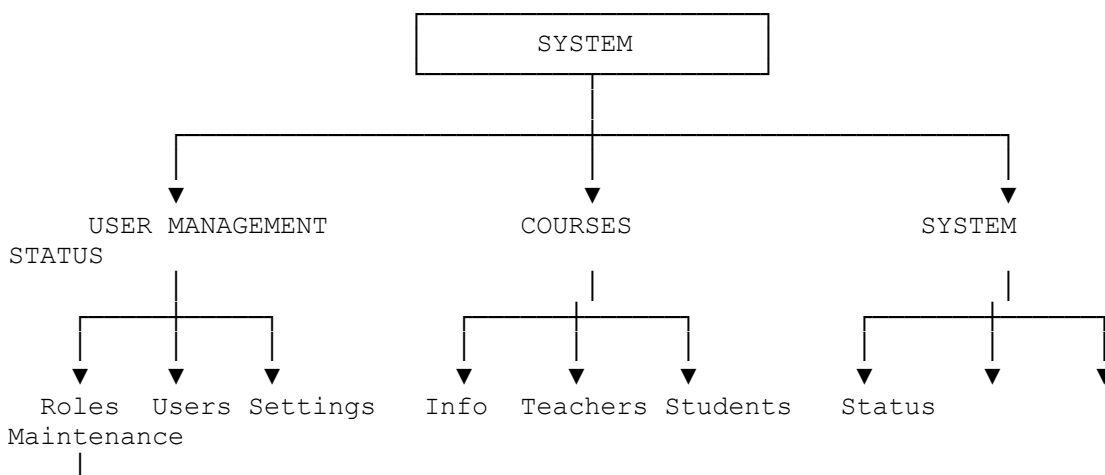
Teacher Module → Running

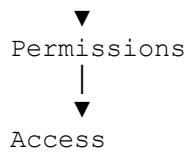
продължават да работят.

Това вече е бъдеща архитектурна стъпка, която ще решим след натрупания опит от V1/V2.

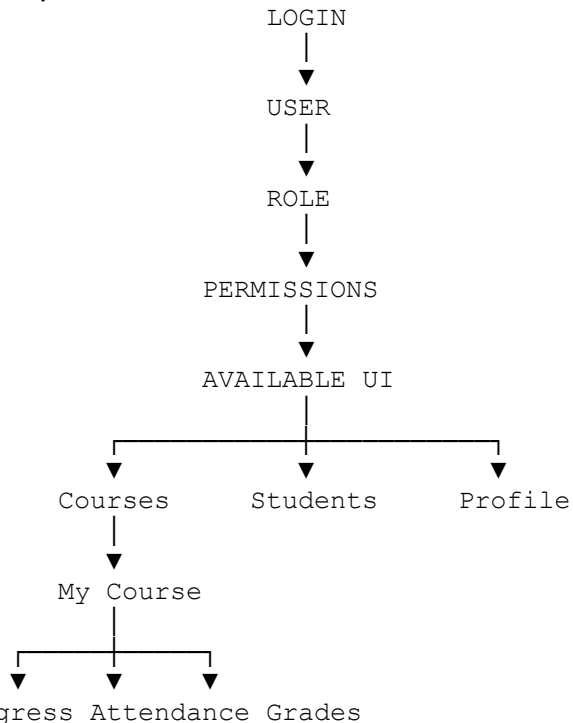
17. Общата блок-схема

Най-кратко цялата система в момента изглежда така:





И потребителският поток:



Финално състояние на архитектурата

В момента имаме доста добра основа:

V1 = Course-centered system / Система, центрирана около курсовете

с:

- 5 роли;
- ясни основни права;
- Course ↔ Student;
- Course ↔ Teacher;
- прогрес;
- присъствие;
- оценки;
- записване;
- UI настройки;
- системен статус;
- проста permission система;
- ясна граница V1/V2/V3.

И най-важното решение, което според мен трябва да запазим като **водещ принцип**:

Не добавяме функционалност само защото технически можем. Добавяме я, когато реално е необходима.

Това ще ни помогне V1 да остане **малка, разбираема, стабилна и лесна за разширяване.**